tf.compat.v1.distribute.OneDeviceStrategy Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/distribute/OneDeviceStrategy

> *A distribution strategy for running on a single device.*

Function Signatures

```
tf.compat.v1.distribute.OneDeviceStrategy( device )
tf.enable_eager_execution() strategy =
tf.distribute.OneDeviceStrategy(device="/gpu:0") with
strategy.scope(): v = tf.Variable(1.0) print(v.device) #
/job:localhost/replica:0/task:0/device:GPU:0 def step_fn(x):
return x * 2 result = 0 for i in range(10): result +=
strategy.run(step_fn, args=(i,)) print(result) # 90
```

Code Examples

```
tf.compat.v1.distribute.OneDeviceStrategy(
device
)
```

```
tf.compat.v1.distribute.OneDeviceStrategy(
device
)
```

```
tf.enable_eager_execution()
strategy = tf.distribute.OneDeviceStrategy(device="/gpu:0")
with strategy.scope():
v = tf.Variable(1.0)
print(v.device)  # /job:localhost/replica:0/task:0/device:GPU:0
def step_fn(x):
return x * 2
result = 0
for i in range(10):
result += strategy.run(step_fn, args=(i,))
print(result)  # 90
```


```
tf.enable_eager_execution()
strategy = tf.distribute.OneDeviceStrategy(device="/gpu:0")
with strategy.scope():
v = tf.Variable(1.0)
print(v.device)  # /job:localhost/replica:0/task:0/device:GPU:0
def step_fn(x):
return x * 2
result = 0
for i in range(10):
result += strategy.run(step_fn, args=(i,))
print(result)  # 90
```

```
os.environ['TF_CONFIG'] = json.dumps({
'cluster': {
'worker': ["localhost:12345", "localhost:23456"],
'ps': ["localhost:34567"]
},
'task': {'type': 'worker', 'index': 0}
})
# This implicitly uses TF_CONFIG for the cluster and current task
info.
strategy = tf.distribute.experimental.MultiWorkerMirroredStrategy()
...
if strategy.cluster_resolver.task_type == 'worker':
# Perform something that's only applicable on workers. Since we set
this
# as a worker above, this block will run on this particular
instance.
elif strategy.cluster_resolver.task_type == 'ps':
# Perform something that's only applicable on parameter servers.
Since we
# set this as a worker above, this block will not run on this
particular
# instance.
```

```
os.environ['TF_CONFIG'] = json.dumps({
'cluster': {
'worker': ["localhost:12345", "localhost:23456"],
'ps': ["localhost:34567"]
},
'task': {'type': 'worker', 'index': 0}
})
# This implicitly uses TF_CONFIG for the cluster and current task
info.
strategy = tf.distribute.experimental.MultiWorkerMirroredStrategy()
...
if strategy.cluster_resolver.task_type == 'worker':
# Perform something that's only applicable on workers. Since we set
this
# as a worker above, this block will run on this particular
instance.
elif strategy.cluster_resolver.task_type == 'ps':
# Perform something that's only applicable on parameter servers.
Since we
# set this as a worker above, this block will not run on this
particular
# instance.
```

```
distribute_datasets_from_function(
dataset_fn, options=None
)
```

```
distribute_datasets_from_function(
dataset_fn, options=None
)
```

Documentation

A distribution strategy for running on a single device.

Inherits From: Strategy

Using this strategy will place any variables created in its scope on the specified device. Input distributed through this strategy will be prefetched to the specified device. Moreover, any functions called via strategy.run will also be placed on the specified device as well.

Typical usage of this strategy could be testing your code with the tf.distribute.Strategy API before switching to other strategies which actually distribute to multiple devices/machines.

For example:

Attributes

In general, when using a multi-worker tf.distribute strategy such as tf.distribute.experimental.MultiWorkerMirroredStrategy or tf.distribute.TPUStrategy(), there is a tf.distribute.cluster_resolver.ClusterResolver associated with the strategy used, and such an instance is returned by this property.

Strategies that intend to have an associated tf.distribute.cluster_resolver.ClusterResolver must set the relevant attribute, or override this property; otherwise, None is returned by default. Those strategies should also provide information regarding what is returned by this property.

Single-worker strategies usually do not have a tf.distribute.cluster_resolver.ClusterResolver, and in those cases this property will return None.

The tf.distribute.cluster_resolver.ClusterResolver may be useful when the user needs to access information such as the cluster spec, task type or task id. For example,

For more information, please see tf.distribute.cluster_resolver.ClusterResolver's API docstring.

Methods

distribute_datasets_from_function

View source

Distributes tf.data.Dataset instances created by calls to dataset_fn.

The argument dataset_fn that users pass in is an input function that has a tf.distribute.InputContext argument and returns a tf.data.Dataset instance. It is expected that the returned dataset from dataset_fn is already batched by per-replica batch size (i.e. global batch size divided by the number of replicas in sync) and sharded. tf.distribute.Strategy.distribute_datasets_from_function does not batch or shard the tf.data.Dataset instance returned from the input function. dataset_fn will be called on the CPU device of each of the workers and each generates a dataset where every replica on that worker will dequeue one batch of inputs (i.e. if a worker has two replicas, two batches will be dequeued from the Dataset every step).

This method can be used for several purposes. First, it allows you to specify your own batching and sharding logic. (In contrast, tf.distribute.experimental_distribute_dataset does batching and sharding

for you.) For example, where experimental_distribute_dataset is unable to shard the input files, this method might be used to manually shard the dataset (avoiding the slow fallback behavior in experimental_distribute_dataset). In cases where the dataset is infinite, this sharding can be done by creating dataset replicas that differ only in their random seed.

The dataset_fn should take an tf.distribute.InputContext instance where information about batching and input replication can be accessed.

You can use element_spec property of the tf.distribute.DistributedDataset returned by this API to query the tf.TypeSpec of the elements returned by the iterator. This can be used to set the input_signature property of a tf.function. Follow tf.distribute.DistributedDataset.element_spec to see an example.

For a tutorial on more usage and properties of this method, refer to the tutorial on distributed input).
If you are interested in last partial batch handling, read this section.

experimental_distribute_dataset

View source

Creates tf.distribute.DistributedDataset from tf.data.Dataset.

The returned tf.distribute.DistributedDataset can be iterated over similar to regular datasets.
NOTE: The user cannot add any more transformations to a tf.distribute.DistributedDataset. You can only create an iterator or examine the tf.TypeSpec of the data generated by it. See API docs of

tf.distribute.DistributedDataset to learn more.

The following is an example:

Three key actions happening under the hood of this method are batching, sharding, and prefetching.

In the code snippet above, dataset is batched by global_batch_size, and calling experimental_distribute_dataset on it rebatches dataset to a new batch size that is equal to the global batch size divided by the number of replicas in sync. We iterate through it using a Pythonic for loop. x is a tf.distribute.DistributedValues containing data for all replicas, and each replica gets data of the new batch size. tf.distribute.Strategy.run will take care of feeding the right per-replica data in x to the right replica_fn executed on each replica.

Sharding contains autosharding across multiple workers and within every worker. First, in multi-worker distributed training (i.e. when you use tf.distribute.experimental.MultiWorkerMirroredStrategy or tf.distribute.TPUStrategy), autosharding a dataset over a set of workers means that each worker is assigned a subset of the entire dataset (if the right tf.data.experimental.AutoShardPolicy is set). This is to ensure that at each step, a global batch size of non-overlapping dataset elements will be processed by each worker. Autosharding has a couple of different options that can be specified using tf.data.experimental.DistributeOptions. Then, sharding within each worker means the method will split the data among all the worker devices (if more than one a present). This will happen regardless of multi-worker autosharding.

By default, this method adds a prefetch transformation at the end of the user provided tf.data.Dataset instance. The argument to the prefetch transformation which is buffer_size is equal to the number of replicas in sync.

If the above batch splitting and dataset sharding logic is undesirable, please use
tf.distribute.Strategy.distribute_datasets_from_function
instead, which does not do any automa